AI Use Appendix
StockTrader Individual Assignment (Coding)

Student: Ariana Rocha (afrocha)
Class: 94815-A4 Agentic Technologies
Assignment: HW2 Building StockTrader

April 13, 2026

Abstract. This appendix describes how I used generative AI tools—**Cursor**, **ChatGPT**, and **Ollama**—while completing the StockTrader coding assignment. I steered work with written instructions and project context, reviewed every stage of output carefully, and used ChatGPT once with the rubric to stress-test the report for gaps. Ollama was required as the local LLM runtime for the assignment pipeline (strategies and evaluator), separate from my authoring workflow. Below I summarize tools, workflow, cleaned representative prompts, sample outputs, and file-level attribution (accepted, revised, or rejected).

1 Tools I used: Cursor, ChatGPT, and Ollama

Cursor Primary environment for repository work, code exploration, edits, and drafting. I created the GitHub repository, added a text file with the assignment description after reading it myself, and layered instructions (e.g., which agents to build, AutoGen + parallel strategies, JSON outputs, no cross-talk before evaluation). Cursor helped scaffold `src/`, `prompts/`, early `README.md` drafts, and initial report-oriented material; I reviewed and revised all of it.

ChatGPT Used later in the process: I pasted my near-final report (or large excerpts) together with the course rubric and asked for a gap analysis—what might be missing, unclear, or weak against the rubric—not for automatic rewriting of the whole document. I incorporated only suggestions I agreed with after manual judgment.

Ollama Part of the assignment itself: local model serving for the three `AssistantAgent` strategies plus the evaluator when running `python -m src.main`. This is runtime behavior of the system, not the same as using a chat UI to draft prose, but it is disclosed here as required AI-assisted inference on market payloads.

2 Workflow: repository, steering file, and review

1. **Read the assignment** on my own, then **captured the brief** in a text file in the repo with any particulars (multi-agent StockTrader, News Sentiment Follower, Volatility Averse, Moral Trader extension, parallel execution, structured JSON, evaluator, optional backtest, Alpha Vantage news path, etc.).

2. **Fed that context into Cursor** so subsequent edits stayed aligned with the spec; I iterated when requirements became clearer (e.g., handoff doc, Alpha Vantage hygiene, report checklist items).

3. **Reviewed meticulously** at each stage: code paths, prompts on disk, `outputs/*.json` after runs, and narrative in report sources. I accepted, revised, or rejected suggestions explicitly rather than bulk-merging blind.

4. **Rubric pass with ChatGPT** after I had a finished draft of the written report: attach rubric + report, ask for missing sections or weak evidence, then I edited the `.tex` / PDF pipeline myself.

3 Representative prompts (cleaned)

The following are paraphrased or lightly edited from what I actually used; they are not verbatim logs.

Repository and build steering (Cursor / project context)

```
I am doing HW2 StockTrader. Here is the assignment text [paste].
Build: AutoGen AssistantAgents for (1) News Sentiment Follower,
(2) Volatility Averse, (3) Moral Trader bonus. They must run in
parallel with structured JSON outputs; no strategy sees another's
output before the evaluator. Use yfinance + optional Alpha Vantage
NEWS_SENTIMENT. Include README, prompts/*.txt on disk, and outputs/
JSON per ticker plus summary.json and backtest bonus.
```

Iteration and hygiene (Cursor)

```
Pull from remote with rebase. Update HANDOFF so API keys only live
in .env; document Alpha Vantage rate limits per official docs.
Adjust market_data for NEWS_SENTIMENT errors and optional sort/limit.
```

Report and documentation (Cursor)

```
Draft README sections for setup (venv, Ollama, .env, PYTHONPATH,
run command). Match the course folder layout. Keep tone honest about
stub mode vs live Ollama for grading.
```

Rubric gap check (ChatGPT)

```
Here is my report draft for StockTrader HW2 [paste or attach].
Here is the rubric [paste]. Grade against the rubric as a TA would:
list gaps, missing quantitative evidence, or unclear claims only.
Do not rewrite the whole report; bullet prioritized fixes.
```

4 Representative outputs (authoring tools; not Ollama runtime)

This subsection is about **generative assistance while building the project and the report**—what came back from **Cursor** and **ChatGPT** when I used the kinds of prompts in the previous section. It is **not** a sample of the StockTrader pipeline's on-ticker LLM outputs (Ollama strategy/evaluator text inside `outputs/*.json`); those are assignment *runtime* inference and are disclosed elsewhere in this appendix. The patterns below are paraphrased or shortened; they show how the tools tended to *continue* a thread—scaffolding, checks, documentation—rather than only mirroring my prompt text.

Repository scaffold and layout (Cursor)

```
Proposed tree: src/main.py CLI, src/orchestration.py with asyncio.gather
for three AssistantAgent calls, prompts/*.txt, outputs/ for JSON.
Add schemas.py for Pydantic shapes; wire evaluator after strategies.
You should trace one ticker end-to-end and confirm no cross-strategy
reads before evaluation.
```

Hygiene and secrets (Cursor; follow-up to HANDOFF / .env work)

```
Keep ALPHAVANTAGE_API_KEY only in repo-root .env (gitignored). Optional
secrets/alphavantage_api_key.txt path is documented in secrets/README.md;
confirm .gitignore covers .env and secrets/. I can suggest a pre-commit
grep for key-like strings; you run it locally before push.
```

README and grader-facing docs (Cursor)

```
Draft bullets: venv activate (PowerShell), PYTHONPATH at repo root,
python -m src.main --backtest. Note STOCKTRADER_SKIP_LLM for CI vs full
Ollama run. Tone: factual, no key material in README.
```

Rubric stress-test (ChatGPT; responds to draft + rubric paste)

```
Prioritized gaps: (1) Tie narrative numbers to a named committed run or
outputs/*.json snapshot. (2) Say explicitly when news_features was empty.
(3) One paragraph on Moral Trader as extension vs graded pair. I am
not rewriting your PDF---address these, then tighten wording yourself.
```

How I used these outputs. I did **not** treat any block as authoritative: I compared suggestions against the assignment, my own code reading, and CLI runs; rejected or rewrote generic or wrong steps; and separated this workflow from **Ollama**, which only runs inside `src/` for strategies and the evaluator when executing the pipeline. If a reply mostly repeated an earlier prompt, I asked for a concrete delta or stopped using that thread.

5 File-level attribution (accept / revise / reject)

Disposition means how the listed artifact left my desk after review: **A** = accepted largely as generated (minor typos only); **R** = revised substantially (logic, safety, or rubric alignment); **X** = rejected initial suggestion and rewrote myself from spec. The rows below mix **R**, **A**, and **X** to reflect different

paths—not a single default. Ollama-generated *text inside* committed JSON is runtime model output, not Cursor-authored source; I still chose what to commit and when.

6 What I verified independently

I treated anything that touches grades or keys as my responsibility, not the model's. Independently (without relying on tool summaries) I confirmed the following by reading code, running the CLI, and opening files:

- **Schema and JSON shape.** Each per-ticker file includes `strategy_a|b|c` with `decision`, `confidence`, `justification`, plus an `evaluator` object with `agents_agree`, `analysis`, and `pattern_note`, matching `src/schemas.py` and the assignment's structured-output requirement.
- **Parallelism and isolation.** In `src/orchestration.py`, the three strategies are awaited together (`asyncio.gather`) on the same market payload; nothing in that phase passes one strategy's completion text into another's system message. The evaluator is only called after all three structured results exist for that ticker.
- **Deterministic routing metadata.** `pattern_note` is derived in Python from the three decisions (`_pattern_note_from_decisions` in `src/evaluator.py`), so the saved note cannot drift from the actual votes even if the LLM paraphrases poorly.
- **Evaluator vs. branch consistency.** When all three decisions match, `agents_agree` is `true`; when the set of labels has more than one value, it is `false`. I stepped through branches in `evaluator_task` and spot-checked tickers where News and Vol disagree.
- **Contradiction guard.** I read the regex blocks in `run_evaluator` that replace evaluator prose when the model claims disagreement under unanimity (or the reverse), and confirmed they fire only on the intended edge cases.
- **Market payload behavior.** `build_market_payload` and Alpha Vantage paths: I verified empty `articles`, null sentiment, and vendor error/rate-limit text surface under `news_features` without crashing the pipeline, and that strategies still receive a single coherent JSON user message.
- **Stub vs. live path.** With `STOCKTRADER_SKIP_LLM=1`, the pipeline returns stub structured objects so the graph can be tested without Ollama; I confirmed that path in `evaluator.py` and strategy code paths as documented in README.
- **Report numbers vs. artifacts.** Tabulated or narrative statistics in `comparative_analysis.tex` were cross-checked against committed `outputs/*.json` and run dates so I did not accidentally describe an old run.
- **Secrets and repo hygiene.** Confirmed API keys are not committed; `.env.example` documents variables; assignment handoff docs describe rate limits from primary vendor documentation.

7 Weak spots, failure modes, and mitigations

Generative components remain the main risk surface. These are the issues I actually saw or explicitly guarded against; they are not hypothetical:

- **Sparse news + strong price/vol fields.** Small models sometimes overweight narrative phrasing ("turbulence", "risk") relative to numeric gates in the JSON, or invent nuance not tied to a field. **Mitigation:** prompt text requires citations to concrete keys; Volatility prompt asks for plain-English lead sentences tied to thresholds; evaluator branch text forces substantive comparison on disagreement.
- **Evaluator prose contradicting arithmetic unanimity.** The model occasionally emitted words like "disagree" when all three `decision` strings matched (or claimed full agreement when they did not). **Mitigation:** post-process replacement strings in `run_evaluator` after structured extraction; `pattern_note` overwritten from code, not the LLM.
- **Extension strategy (Moral) variability.** Third philosophy is useful but less constrained by external data; confidence and justification can look mismatched (e.g. low confidence with assertive wording). **Mitigation:** kept scope clearly "same JSON" and relied on evaluator to contrast philosophies without treating Moral as ground truth.
- **Vendor fragility.** Alpha Vantage rate limits and error payloads appear inside `news_features.note`; strategies must not treat that blob as tradable news. **Mitigation:** documented in `docs/ALPHA_VANTAGE.md`; optional redaction script under `scripts/` for committed samples.
- **Committed JSON drift.** Any snapshot in `outputs/` can become stale relative to code after refactors. **Mitigation:** re-ran pipeline when logic changed; verified evaluator and strategy keys before final submission.

Artifact	Summary (tooling touch)	Disp.
`src/orchestration.py`, `src/main.py`	Parallel `asyncio.gather` for strategies; per-ticker flow; Cursor-assisted scaffold, manual trace of call order	R
`src/market_data.py`	Rejected early draft that obscured vendor failures; rewrote error handling after Alpha Vantage docs	X
`src/evaluator.py`	Branching evaluator task text; deterministic `pattern_note`; regex guard on contradictory "agree/disagree" phrasing in `analysis`	R
`src/schemas.py`, `src/llm_factory.py`, `src/strategies.py`	Pydantic shapes and prompt loading; thin glue modules—accepted scaffold, minor typing/import fixes only	A
`src/backtest.py`	Replaced first Cursor heuristic with formulas aligned to assignment intent	X
`prompts/*.txt`	Rejected generic first drafts; rewrote all four for structured outputs and rubric tone	X
`README.md`, `src/README.md`	Cursor setup draft; accepted after command-by-command verification and tone trim	A
`docs/*.md` (e.g. `ALPHA_VANTAGE.md`, `HANDOFF_NEXT_AGENT.md`)	Operational notes; fact-checked against primary sources; light copy edits	R
`report/comparative_analysis.tex`	Reorganized report outline and TikZ against committed `outputs/`; not the first AI-suggested structure	X
`report/ai_use_appendix.tex`	Disclosure prose authored by hand; LaTeX table formatting iterated locally	R
`report/report_checklist.md`	Personal checklist; occasional wording nits only	R
`report/LATEX_BUILD.md`	One-page build commands; accepted as-is after path check	A
`report/walkthrough/*.html`, `report/walkthrough/walkthrough.css`	Map, canvases, market-data page; iterated layout/CSS for grader clarity	R
`report/walkthrough/Walkthrough_Artifacts.html`	Outputs walkthrough page; split from map for readability; refined SVG + file cards locally	R
`scripts/*.py`	Redaction / maintenance scripts; small, reviewed line-by-line; kept nearly verbatim	A
`outputs/*.json`	Produced by `python -m src.main` (Ollama or stub); I diffed keys, spot-checked decisions vs. payload, chose commit set for grading	Runtime